RAG-BASED CUSTOMER SUPPORT ASSISTANT

High-Level Design Document

Prepared by: Rahul Pravin Dhumal

Intern ID: IN226095102

Document Type	High-Level Design (HLD)
Project Title	RAG-Based Customer Support Assistant
Modules	(loader, embedder, retriever, graph, hitl, memory, main, config)
Workflow Engine	LangGraph (3-node graph) Process Node → Answer Node / HITL Node
API	FastAPI — POST /ask

1. System Overview

1.1 Problem Statement :

Customer support is a critical business function that is traditionally slow, expensive, and heavily dependent on human agents. Users seeking help with orders, refunds, and account issues often face long wait times, inconsistent answers, and limited availability outside business hours.

Key challenges with traditional support:

- High response latency — users wait minutes or hours for answers
- Agent inconsistency — different agents give different answers
- Scalability bottleneck — human agents cannot handle traffic spikes
- Knowledge silos — support documentation is underutilized
- No 24/7 availability without enormous cost

1.2 Proposed Solution :

We designed a RAG-based Customer Support Assistant that combines document retrieval, large language models, and graph-based workflow orchestration to create a scalable, intelligent support system.Uses a PDF knowledge base

- Retrieves relevant information
- Generates contextual answers
- Uses workflow-based decision making
- Escalates complex queries to human support (HITL)

1.3 Scope of the System :

In Scope	PDF ingestion, chunk embedding, vector storage, query retrieval, LLM response generation, LangGraph workflow, HITL escalation, REST API
Out of Scope	Multi-document ingestion, real-time chat UI, user authentication, analytics dashboard (planned for future iterations)
Deployment	Local development with FastAPI — containerized deployment planned

2. System Architecture

2.1 Architecture Overview :

The system is composed of two major pipelines: a Document Ingestion Pipeline (offline) and a Query Processing Pipeline (online). Both pipelines feed into a shared Vector Store and are orchestrated by the LangGraph Workflow Engine.

2.2 Architecture Diagram :

```
DOCUMENT INGESTION PIPELINE (Offline)

[ PDF Knowledge Base ]
↓
[ Document Loader (PyPDFLoader) ]
↓
[ Text Chunker (RecursiveCharacterTextSplitter) ]
↓
[ Embedding Model (HuggingFace: all-MiniLM-L6-v2) ]
↓
[ ChromaDB Vector Store ]

QUERY PROCESSING PIPELINE (Online)

[ User Query via FastAPI / Swagger UI ]
↓
[ LangGraph Workflow Engine ]
↓
[ Retriever Node — ChromaDB Similarity Search ]
↓
[ Routing Layer — Intent Detection + Score Check ]
↓                    ↓
[ LLM Node (Groq) ]  [ HITL Escalation Node ]
↓                    ↓
[ Final Response ]   [ human_review_queue.txt ]
```

3. Major Component Descriptions

Component	Technology Used	Purpose
User Interface	FastAPI + Swagger UI	Entry point for user queries; future React frontend planned
Document Loader	PyPDFLoader (LangChain)	Reads and parses the PDF knowledge base into raw text
Text Chunker	RecursiveCharacterTextSplitter	Splits documents into overlapping chunks for better retrieval
Embedding Model	HuggingFace all-MiniLM-L6-v2	Converts text into dense 384-dimensional vector embeddings
Vector Store	ChromaDB	Persists embeddings; enables fast cosine similarity search
Retriever	ChromaDB Retriever + Reranker	Fetches top-k relevant chunks; reranks by relevance score
LLM	Groq (llama-3.3-70b-versatile)	Generates contextual answers using retrieved chunks as context
Workflow Engine	LangGraph	Orchestrates node execution, state transitions, and routing
Routing Layer	Custom Logic (Python)	Decides between answer node and HITL based on confidence
HITL Module	File-based Queue	Logs escalated queries to human_review_queue.txt for review
Memory Module	Custom in-memory conversation buffer	Stores previous conversation turns and enables memory-aware routing for handling follow-up queries.

4. Data Flow (End-to-End)

4.1 Document Ingestion Flow (Offline)

1. **Load PDF** : PyPDFLoader reads the customer support PDF and creates Document objects with page content and metadata

2. **Chunk text** : RecursiveCharacterTextSplitter splits documents into 500-character chunks with 50-character overlap to preserve context at boundaries

3. **Generate embeddings** : HuggingFace all-MiniLM-L6-v2 converts each chunk into a 384-dimensional vector representing its semantic meaning.

4. **Store in ChromaDB** : Embedding vectors and their associated text chunks are persisted in ChromaDB for fast similarity lookup at query time.

4.2 Query Processing Flow (Online)

5. **Receive query via API** : User submits a query through FastAPI. The request is validated and passed to the LangGraph workflow engine

6. **Retrieve relevant chunks** : The Retriever queries ChromaDB with the embedded user query and returns the top-k most similar document chunks.

7. **Apply reranking** : Retrieved chunks are scored and reranked by relevance. The overall confidence score is computed based on top similarity scores.

8. **Send context to LLM** : The routing layer evaluates the confidence score and query intent. High-confidence queries proceed to the LLM node; low-confidence queries route to HITL.

9. **Generate response** : Groq's llama3-8b-8192 model receives the retrieved context, conversation memory, and user query as a structured prompt. It generates a contextual answer.

10. **HITL Escalation (if triggered)** : Complex or low-confidence queries are logged to human_review_queue.txt with full context for human agent review and response.

11. **Return Response** : The final answer (LLM-generated or HITL acknowledgment) is returned to the user via the FastAPI response.

5.Technology Choices

All technology choices were made considering performance, cost, local deployment requirements, and alignment with the project's use case.

Technology	Reason for Choice	Alternatives Considered
ChromaDB	Lightweight, easy local setup, fast cosine similarity search, no external service dependency	Pinecone (cloud), Weaviate, FAISS
LangGraph	Supports workflow-based AI with stateful graphs, enables conditional routing, required by project spec	LangChain LCEL, Haystack Pipelines
Groq (llama3-8b-8192)	Extremely fast inference (< 1s), free tier available, strong LLM quality for customer support tasks	OpenAI GPT-4, Ollama (local), Mistral
HuggingFace Embeddings	Completely free, runs offline, all-MiniLM-L6-v2 provides strong semantic similarity performance	OpenAI Embeddings (paid), Cohere
FastAPI	High-performance Python API framework, auto-generates Swagger UI, async support	Flask, Django REST Framework
LangChain	Rich ecosystem of document loaders, text splitters, and retrievers that integrate with all components	LlamaIndex, Custom implementation

6. Scalability Considerations

While the current implementation targets local deployment for demonstration purposes, the architecture was intentionally designed to scale. The following table outlines anticipated challenges and the corresponding design decisions that address them.

Large PDF Documents	Chunking strategy (500 chars + 50 overlap) ensures any document size is handled. ChromaDB collections can hold millions of vectors without performance degradation.
High Query Load	FastAPI's async request handling supports concurrent queries. The retrieval step is a fast vector search (milliseconds). LLM calls can be batched or load-balanced across Groq API keys.
Latency Concerns	Groq's hardware-accelerated inference reduces LLM response time to under 1 second. ChromaDB's ANN (approximate nearest neighbor) search returns chunks in under 50ms.

7. HLD Summary

This High-Level Design describes a complete, production-grade Retrieval-Augmented Generation system for customer support. The system integrates the following capabilities into a cohesive architecture

RAG Architecture	PDF ingestion → chunking → embedding → vector retrieval → LLM generation
Workflow Orchestration	LangGraph manages state, routing, and node execution as a directed graph
Conditional Routing	Confidence scores and intent detection determine answer vs. escalation paths
HITL Integration	Low-confidence and complex queries are escalated to human review queue
Technology Stack	ChromaDB + Groq + HuggingFace Embeddings + LangChain + FastAPI
Scalability	Designed to scale from local demo to cloud production deployment